Andrew J. Anagnost College of Engineering & Computer Science

CSUN. CALIFORNIA STATE UNIVERSITY NORTHRIDGE

COMP 642 – Advanced Databases and Data Visualization

EventSphere: Multi-Database Event Ticketing and Discovery System

Professor: Dr. Alex Modarresi

System

Prepared by:

Daniel Graham

Jean Paul Collazo

Janeth Lucero Garcia Rodriguez

October 9, 2026

Los Angeles, California

Table of Contents

1. Introduction 5
2. Requirements 6
   2.1 Requirements Overview 6
   2.2 Application Functionality Requirements 6
   2.3 Relational Database Requirements 6
   2.4 MongoDB Requirements 6
   2.5 Redis Requirements 6
   2.6 Application Integration Requirements 7
   2.7 Cross-Database Operation Requirements 7
   2.8 Performance Experiment Requirements 7
   2.9 Scope and Acceptance Criteria 7
3. System Architecture 8
   3.1 Architectural Overview 8
   3.2 Containerized Deployment Architecture 8
   3.3 FastAPI Application Layer 8
   3.4 Database Responsibilities and Boundaries 8
   3.5 Cross-Database Event-Detail Flow 9
   3.6 Frontend and Reverse-Proxy Integration 9
   3.7 Transaction Boundary and Failure Behavior 9
   3.8 Architectural Assessment 10
4. Relational Database Design 10
   4.1 Role of PostgreSQL 10
   4.2 Relational Entity-Relationship Model 10
   4.3 Relational Schema 11
   4.4 Indexing Strategy 12
   4.5 Seed Data 12
   4.6 Required SQL Queries 12
      4.6.1 Query 1: Events at a Particular Venue 12
      4.6.2 Query 2: Tickets Purchased by a Particular User 13
      4.6.3 Query 3: Total Tickets Sold for Each Event 13
      4.6.4 Query 4: Remaining Ticket Inventory 13
      4.6.5 Query 5: Total Revenue for Each Event 14
      4.6.6 Query 6: Customers Who Purchased the Most Tickets 14

4.6.7 Query 7: Events Above a Sales Threshold 15
4.6.8 Query 8: Monthly Ticket Revenue 15
4.7 Transaction Processing 15
4.8 Transaction Demonstration 16
4.9 Monetary Data and Relational Limitations 16
4.10 PostgreSQL Schema Evidence 16
5. MongoDB Design 17
5.1 Role of MongoDB 17
5.2 MongoDB Document Model 17
5.3 Collection Initialization and Shared Identifiers 17
5.4 Required MongoDB Operations 17
5.5 Cross-Database Retrieval and Mutations 18
5.6 Design Justification and Limitations 18
6. Redis Design 19
6.1 Role and Configuration 19
6.2 Event-Response Cache 19
6.3 Trending Sorted Set 19
6.4 Demonstration and Performance 19
6.5 Consistency, Reliability, and Limitations 19
7. Application Integration 20
7.1 FastAPI Application Boundary 20
7.2 Angular API Playground 20
7.3 Reverse-Proxy and Container Integration 20
7.4 End-to-End Request Flow 20
7.5 Integration Assessment 21
8. Cross-Database Operation 21
8.1 Composite Event Retrieval 21
8.2 Cross-Database Ownership and Mutations 21
8.3 Consistency Boundary 21
9. Performance Experiment 21
9.1 Experiment Design 21
9.2 Results and Interpretation 22
9.3 Validity and Reproducibility 22
10. Architecture Analysis 22

10.1 Architectural Strengths 22
10.2 Trade-Offs and Scalability 22
10.3 Security and Deployment Readiness 22
11. Challenges and Limitations 22
12. Conclusions 23

1. Introduction

Event ticketing platforms must manage several types of information and workload patterns at the same time. Structured records such as users, events, venues, orders, ticket items, and payments require consistency, relationships, and transactional behavior. At the same time, event content such as speakers, schedules, reviews, descriptions, and tags may vary depending on the type of event. Frequently requested event information and popularity rankings also benefit from low-latency access.

EventSphere is a containerized event discovery and ticketing prototype designed to demonstrate how multiple database technologies can support different responsibilities within one application. The system uses PostgreSQL for structured transactional information, MongoDB for flexible event-content data, and Redis for event caching and trending-event scores.

The PostgreSQL database contains the core relational entities required by the project, including users, venues, events, ticket types, orders, order items, and payments. These tables support event discovery, ticket-purchase records, order history, inventory calculations, revenue analysis, and customer-purchase analysis. The relational schema and seed data are defined in the project's postgres/init.sql file.

MongoDB stores additional event information in the group4db.event_content collection. These documents contain flexible fields such as descriptions, speakers, schedules, reviews, and tags. The documents intentionally use nested objects and arrays to demonstrate the advantages of a document-oriented model. The collection and its initial data are defined in mongo/mongo-init.js.

Redis supports two performance-oriented use cases. First, it caches combined event responses using keys such as event:{event_id} with a 60-second expiration period. Second, it maintains the trending:events sorted set, which ranks events according to successful event-detail views. These operations are coordinated by the FastAPI event routes in api/services/events_router.py.

The application layer connects the databases through FastAPI. The application retrieves structured event information from PostgreSQL, retrieves flexible content from MongoDB, optionally uses Redis for caching, and combines the results into a single response. The overall environment is defined through docker-compose.yml. The API entry point is located in api/main.py, while the Angular API Playground is implemented in web/src/app/app.ts.

The project's objective is not to provide a production-ready commercial ticketing platform. Instead, it demonstrates how PostgreSQL, MongoDB, Redis, FastAPI, Angular, Nginx, and Docker can be integrated into a coherent multi-database architecture. The project uses synthetic data and a local containerized deployment. In addition, the current transaction demonstrates atomic creation of orders and order items, while payment processing, inventory updates, enterprise authentication, and production-scale deployment remain outside the project scope. The order transaction is implemented in api/services/orders_router.py.

This report presents the system requirements, architecture, relational database design, MongoDB design, Redis implementation, application integration, cross-database operation, performance experiment, architectural justification, limitations, and conclusions.

2. Requirements

2.1 Requirements Overview

EventSphere is required to demonstrate an integrated event ticketing and discovery system using PostgreSQL, MongoDB, Redis, a FastAPI application layer, and an Angular API Playground. The requirements address application functionality, database operations, cross-database integration, deployment, and performance evaluation. The complete containerized environment is defined in docker-compose.yml.

The project uses a polyglot persistence approach. PostgreSQL manages structured transactional data, MongoDB manages flexible event-content information, Redis provides caching and trending-event functionality, and FastAPI coordinates requests between the databases. The requirements are organized into application functionality, relational database operations, MongoDB operations, Redis functionality, application integration, cross-database retrieval, performance experimentation, and architectural analysis.

2.2 Application Functionality Requirements

The system must allow users to interact with event information and demonstrate the principal backend operations through HTTP endpoints. The available operations include listing events, retrieving an individual event, retrieving MongoDB event content, searching by tags, speakers, review ratings, nested attributes, and genre, adding and deleting reviews, listing and creating users, retrieving a user's orders, creating and retrieving orders, retrieving trending event scores, and executing SQL statements. The FastAPI application registers the event, user, order, and SQL routers in api/main.py.

The system is intended to demonstrate database functionality rather than provide a complete commercial marketplace. A separate role-based administrative portal, production authentication system, and external payment processor are outside the current scope. Although the data model includes administrative attributes, the current prototype does not enforce role-based access control.

2.3 Relational Database Requirements

PostgreSQL must provide the authoritative system of record for structured and transactional data. The relational design must contain Users, Events, Venues, TicketTypes, Orders, OrderItems, and Payments, together with primary keys, required attributes, unique constraints where appropriate, relational joins, indexes, and representative seed data. The schema and initial data are defined in postgres/init.sql.

The relational database must support eight analytical operations: finding events at a venue; finding tickets purchased by a user; calculating confirmed tickets sold by event; calculating remaining inventory; calculating completed revenue; identifying customers who purchased the most tickets; finding events above a sales threshold; and grouping completed revenue by month.

The system must also demonstrate transaction processing for order creation. The order and its order items must be added through one SQLAlchemy session, committed together when successful, and rolled back if an exception occurs. This behavior is implemented in api/services/orders_router.py.

2.4 MongoDB Requirements

MongoDB must store flexible event-content information that can vary according to the type of event. The implementation must demonstrate insertMany(), insertOne(), a unique index on eventId, nested descriptive objects, arrays of speakers, schedules, reviews, and tags, projections, comparison operators, Boolean operators, dot notation, and $elemMatch. It must also demonstrate updates using $push and deletes using $pull. The collection, seed documents, and index are defined in mongo/mongo-init.js, while the retrieval and mutation endpoints are implemented in api/services/events_router.py.

The MongoDB design must remain separate from PostgreSQL transactional ownership. MongoDB enriches an event with flexible content, but it does not become the authority for ticket inventory, order status, payment status, or confirmed sales.

2.5 Redis Requirements

Redis must support two performance-oriented use cases implemented in the FastAPI event routes in api/services/events_router.py. First, the application must cache complete event responses under keys using the pattern event:{event_id} with a 60-second time-to-live. The event endpoint must provide a bypass_cache=true option so that database-backed requests can be compared with cached requests. Second, Redis must maintain the trending:events sorted set, where each event UUID is a member and its score represents successful event-detail views. The API must increment scores with ZINCRBY and return the top ten members using ZREVRANGE.

Redis is a supporting data store and must not replace PostgreSQL as the authority for orders, payments, or ticket inventory. Redis failure may reduce performance or disable the trending response, but event retrieval should remain capable of using PostgreSQL and MongoDB.

2.6 Application Integration Requirements

The FastAPI layer must coordinate communication between the client and the three databases. The client must not connect directly to PostgreSQL, MongoDB, or Redis. The backend must read connection information from environment variables, use the appropriate database for each data responsibility, combine PostgreSQL and MongoDB data into event responses, optionally cache those responses in Redis, and return JSON through REST endpoints. The router registration and API entry point are defined in api/main.py.

The Angular API Playground must provide editable path parameters, query parameters, and JSON request bodies. It must display the HTTP status, response payload, and elapsed request time for each request. The frontend implementation is defined in web/src/app/app.ts. The frontend communicates through the /api path, which is routed to FastAPI by the Nginx configuration inside the web container.

The completed interface organizes the available endpoints into Events, Users, Orders, and Database groups. The current implementation exposes 17 endpoint cards in total.

2.7 Cross-Database Operation Requirements

The system must demonstrate a cross-database event-detail operation. When a client requests an event, FastAPI checks Redis for a cached response unless caching is bypassed. PostgreSQL provides the structured event record, MongoDB provides the matching event-content document using the shared UUID, and FastAPI combines the results into one response. When caching is enabled, the combined response is stored in Redis, and the event's trending score is incremented when the event exists.

This operation demonstrates application-level integration rather than a direct database join. The databases remain independently managed, while FastAPI coordinates the shared identifier, retrieval sequence, cache behavior, and response composition.

2.8 Performance Experiment Requirements

The project must compare event-detail response times with Redis caching enabled and with Redis bypassed. The experiment must include ten requests using the normal cached endpoint, ten requests using bypass_cache=true, recorded response times, minimum, maximum, average, and median values, an explanation of the performance difference, and a discussion of caching benefits, invalidation concerns, and time-to-live selection. The detailed methodology, raw measurements, calculations, and visualization are presented in Section 9.

2.9 Scope and Acceptance Criteria

The project satisfies the requirements when the implementation and report demonstrate the following outcomes:

Requirement area	Acceptance condition
PostgreSQL	Seven required entities, seed data, indexes, eight SQL queries, and transactional order creation are documented and implemented.
MongoDB	Flexible event-content documents, required operators, indexing, updates, deletes, and searches are demonstrated.
Redis	Event caching with a 60-second TTL and a top-ten trending sorted set are implemented.
FastAPI	The backend coordinates the databases and exposes the required REST operations.
Angular	The API Playground submits requests and displays status, payload, and elapsed time.
Cross-database operation	PostgreSQL and MongoDB data are combined through FastAPI and optionally cached in Redis.
Performance	Cached and uncached requests are compared using ten-call experiments.
Architecture analysis	Data ownership, design justification, limitations, and trade-offs are explained.

Table 1. Requirements Acceptance Criteria

The project is a local Dockerized prototype using synthetic data. Production-scale security, high availability, payment processing, role-based access control, multi-region deployment, and advanced inventory reservation are outside the acceptance scope.

3. System Architecture

3.1 Architectural Overview

EventSphere uses a containerized, layered architecture based on polyglot persistence. A browser client interacts with the Angular API Playground, which is served by Nginx in the web container. Requests sent through the /api path are forwarded to the FastAPI application. FastAPI acts as the application boundary: it validates requests, invokes the appropriate repository or database client, combines data when required, applies cache behavior, and returns JSON responses to the client.

The deployment is defined in docker-compose.yml. The Compose configuration defines five services: local-postgres, local-mongodb, local-redis, comp642-api, and comp642-web. The first three services provide the databases, the API service provides the FastAPI backend, and the web service provides the Angular build and Nginx reverse proxy. This arrangement keeps the client, application layer, and data stores separated while allowing the complete prototype to be started as one local environment.

The architecture is intentionally organized around data ownership. PostgreSQL is the authoritative store for structured entities and transaction-oriented records. MongoDB stores flexible event-content documents. Redis stores derived or temporary information used to improve response time and rank event views. The application, rather than any single database, coordinates the cross-database event-detail response.

3.2 Containerized Deployment Architecture

The local deployment uses Docker Compose service names as internal hostnames. The FastAPI container connects to PostgreSQL through local-postgres, to MongoDB through local-mongodb, and to Redis through local-redis. These names are supplied through environment variables, so the application does not need to hard-code localhost addresses for communication between containers.

The PostgreSQL service exposes port 5432 and initializes the group4db database from the PostgreSQL initialization script. MongoDB exposes port 27017 and receives its seed script through the Docker initialization directory. Redis exposes port 6379 and runs with append-only persistence enabled. The API container exposes port 8890 and starts Uvicorn on 0.0.0.0, while the web container exposes the browser-facing application on port 8096.

The Compose file also declares persistent volumes for MongoDB, Redis, and PostgreSQL. These volumes preserve database state across container restarts, but they also mean that initialization scripts may not execute again when an existing volume is reused. Consequently, a clean initialization or direct verification of the current database state is necessary when preparing evidence for the report.

The service dependencies establish the intended startup order: the API depends on the three database services, and the web service depends on the API. The current configuration does not define health checks, so depends_on should be understood as an orchestration order rather than proof that every database is ready to accept connections. This is suitable for the local demonstration but is a limitation for a production deployment.

3.3 FastAPI Application Layer

The FastAPI application is created and configured in api/main.py. The application registers four routers: events, users, orders, and SQL execution. It also configures CORS for the local frontend origins and exposes the FastAPI documentation endpoints. This central entry point gives the client one HTTP boundary while allowing each router to focus on a separate functional area.

The event router is the principal integration component. It obtains structured events through the PostgreSQL repository layer, queries MongoDB for event content, reads and writes Redis cache entries, and maintains the trending sorted set. The implementation is located in api/services/events_router.py. This router therefore demonstrates both database-specific operations and application-level coordination across the three data stores.

The order router uses the PostgreSQL DBContext and repository classes to create and retrieve orders. Its create-order endpoint uses one SQLAlchemy session for the order and its order items, commits the session when all records are staged successfully, and rolls back the session if an exception occurs. The implementation is located in api/services/orders_router.py.

3.4 Database Responsibilities and Boundaries

PostgreSQL owns the structured business records: users, events, venues, ticket types, orders, order items, and payments. This responsibility is appropriate because those records require typed columns, relational joins, indexes, and transaction support. The repository layer provides the application with a consistent way to retrieve and persist these records without exposing SQLAlchemy sessions directly to the frontend.

MongoDB owns event enrichment rather than transactional purchase state. The event_content collection stores a shared event identifier together with nested descriptions, speakers, schedules, reviews, and tags. MongoDB is accessed through the FastAPI layer, which uses standard UUID representation so that the MongoDB eventId can be matched with the PostgreSQL events.id value. This is an application-managed relationship rather than a database-enforced foreign key.
Redis owns two derived workloads. The event response cache stores a serialized combination of PostgreSQL and MongoDB data under an event-specific key with a 60-second time-to-live. The trending:events sorted set stores event identifiers and view scores. Redis therefore improves access speed and provides ranking behavior, but it is not the source of truth for orders, payments, confirmed sales, or ticket inventory.

3.5 Cross-Database Event-Detail Flow

The main cross-database operation is the event-detail request, GET /events/{id}. The operation illustrates how independent databases can be combined through an application service rather than through a direct database join. The request proceeds through the following sequence:

- FastAPI constructs the Redis key event:{id} and checks the cache unless bypass_cache=true is supplied.
- When a valid cached response is found, FastAPI parses the serialized event response, increments the event's trending score, and returns the response without querying PostgreSQL or MongoDB.
- On a cache miss or a bypassed request, FastAPI uses the PostgreSQL repository layer to retrieve the structured event record.
- If the relational event exists, FastAPI queries MongoDB using the same UUID in standard BSON UUID representation and attaches the matching event-content document.
- When caching is enabled, FastAPI serializes the combined response and stores it in Redis for 60 seconds. The event's trending score is incremented only when the event exists.
- FastAPI returns the composed JSON response to the client. A Redis failure during retrieval, cache population, or trending-score increment is treated as a degraded supporting-service condition, while the database-backed event response can continue when possible.

This sequence creates a clear separation between data retrieval and response composition. PostgreSQL and MongoDB remain independently managed, while FastAPI controls the shared identifier, retrieval order, cache policy, and response contract. The result is a single event response that contains both relational event fields and flexible event content without duplicating all content fields in the relational schema.

3.6 Frontend and Reverse-Proxy Integration

The Angular API Playground is implemented in web/src/app/app.ts. It presents the available operations as endpoint cards and allows the user to edit path parameters, query parameters, and JSON request bodies. For each request, the interface displays the HTTP status, response payload, and elapsed time. This makes the database operations demonstrable through the running application rather than only through source code or command-line clients.
Nginx serves the compiled Angular application and forwards requests beginning with /api/ to the FastAPI service on port 8890. The browser therefore communicates with one public web endpoint on port 8096, while the frontend does not connect directly to PostgreSQL, MongoDB, or Redis. During development, the Angular proxy configuration provides the equivalent /api forwarding behavior; in the containerized deployment, the Nginx configuration performs the reverse-proxy function.
This arrangement improves separation of concerns. Angular is responsible for interaction and demonstration, Nginx is responsible for serving static files and forwarding API traffic, FastAPI is responsible for validation and orchestration, and the databases are responsible for persistence according to their assigned data models.

3.7 Transaction Boundary and Failure Behavior

The architecture contains a local transaction boundary for relational order creation, but it does not implement a distributed transaction across PostgreSQL, MongoDB, and Redis. The POST /orders operation commits the order and order items together within one PostgreSQL session. It does not create a payment record, update MongoDB content, or update Redis inside that transaction. This boundary is appropriate for the current demonstration because order composition is the implemented atomic operation, while cross-database event retrieval is a read-and-compose workflow.
Failure handling follows the importance of each component. PostgreSQL failures affect the structured event and order operations. MongoDB failures affect event-content retrieval, search, and review mutation operations. Redis failures normally degrade event retrieval performance or disable trending rather than invalidate the authoritative PostgreSQL data. The /trending endpoint is more dependent on Redis and returns an error when the sorted set cannot

be retrieved. These behaviors make the system partially fault tolerant, but they do not provide high availability or automatic failover.

3.8 Architectural Assessment

The architecture is well suited to the project because each database is used for a workload that matches its strengths. PostgreSQL provides structure and transactional integrity, MongoDB provides flexible nested event content, and Redis provides low-latency derived data. FastAPI supplies the coordination point that connects these technologies without requiring the frontend to understand the internal database topology.

The primary architectural trade-off is that consistency across databases is managed by the application. A shared UUID connects PostgreSQL and MongoDB records, but the databases cannot enforce the relationship together. In addition, cached event responses can remain temporarily stale after MongoDB review updates because the current implementation relies on the 60-second expiration rather than explicit event-cache invalidation. These trade-offs are acceptable for the prototype and are examined further in the MongoDB, Redis, architecture-analysis, and limitations sections.

Overall, EventSphere demonstrates a coherent local multi-database architecture: a browser-facing Angular and Nginx layer, a FastAPI orchestration layer, and three specialized data stores connected through Docker Compose. The design establishes the foundation for the detailed database discussions in Sections 4 through 6 and the application, cross-database, and performance analysis that follows.

4. Relational Database Design

4.1 Role of PostgreSQL

PostgreSQL serves as the authoritative system of record for EventSphere. It stores structured information that requires defined relationships, consistent data types, relational joins, and transactional behavior. The schema and seed data are defined in postgres/init.sql.

The PostgreSQL database contains the seven required entities:

- Users
- Events
- Venues
- TicketTypes
- Orders
- OrderItems
- Payments

These entities support account management, event discovery, ticket purchasing, order history, payment records, inventory calculations, and sales analysis. PostgreSQL is also responsible for the analytical queries required by the project. These queries use joins, aggregation, filtering, grouping, ordering, and HAVING conditions to answer operational questions about event activity and ticket sales.

4.2 Relational Entity-Relationship Model

The logical relational model can be summarized as follows. Venues host Events; Users place Orders for Events; Orders contain OrderItems; OrderItems reference TicketTypes; and Orders are associated with Payments. The Users-to-Events many-to-many relationship is represented through Orders, which acts as the transactional bridge between the customer and the event.

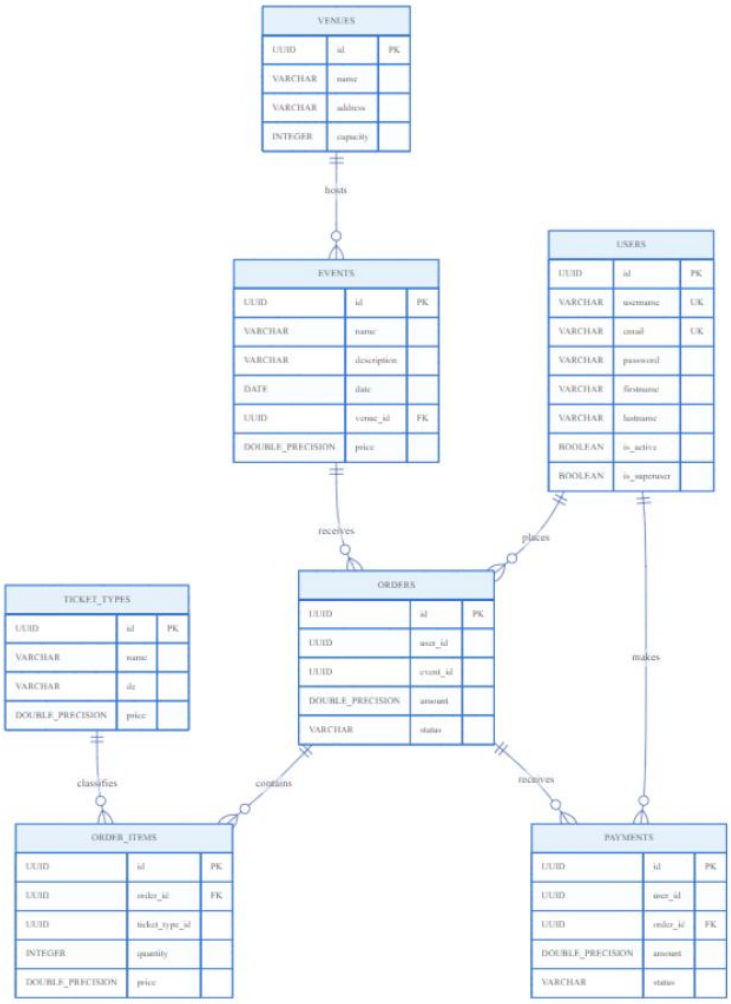


Figure 1. EventSphere relational entity-relationship diagram.

The current schema explicitly enforces some foreign keys, including events.venue_id referencing venues.id, order_items.order_id referencing orders.id, and payments.order_id referencing orders.id. The relationships involving orders.user_id, orders.event_id, order_items.ticket_type_id, and payments.user_id are represented in the application and logical model but should receive explicit PostgreSQL foreign-key constraints for stronger referential integrity.

4.3 Relational Schema

Entity	Important attributes	Constraints and purpose
Users	id, username, email, password, firstname, lastname, is_active, is_superuser	UUID primary key; unique username and email; required account fields
Venues	id, name, address, capacity	UUID primary key; capacity supports inventory calculations
Events	id, name, description, date, venue_id, price	UUID primary key; venue relationship; event date and price
TicketTypes	id, name, description, price	UUID primary key; ticket category and price information
Orders	id, user_id, event_id, amount, status	UUID primary key; identifies a customer purchase
OrderItems	id, order_id, ticket_type_id, quantity, price	UUID primary key; associates ticket quantities with an order
Payments	id, user_id, order_id, amount, status	UUID primary key; records payment amount and status

Table 2. PostgreSQL Entity Definitions

The schema uses UUID values as identifiers. UUIDs are appropriate for this application because they provide globally unique identifiers that can be shared across PostgreSQL, MongoDB, Redis, and the application layer. The event-content documents in MongoDB use the same event UUID as the PostgreSQL event record. This allows the application layer to retrieve relational event information and flexible event content using one shared identifier.

4.4 Indexing Strategy

The schema defines indexes for columns frequently used in joins, filters, and application lookups.

Index	Table and column	Intended use
idx_orders_user_id	orders.user_id	Retrieve a user's orders
idx_orders_event_id	orders.event_id	Retrieve orders for an event
idx_events_venue_id	events.venue_id	Find events at a venue
idx_order_items_order_id	order_items.order_id	Retrieve items associated with an order
idx_order_items_ticket_type_id	order_items.ticket_type_id	Locate items by ticket type
idx_payments_order_id	payments.order_id	Retrieve payment information for an order

Table 3. PostgreSQL Indexes

The indexes support the most common relationship-based queries. Primary keys and unique constraints also create supporting indexes automatically in PostgreSQL. For a production implementation, additional composite or partial indexes could be considered for frequent analytical filters such as confirmed orders or completed payments. The current project uses a small synthetic dataset, so the primary objective is to demonstrate appropriate indexing decisions rather than large-scale query optimization.

4.5 Seed Data

The database is initialized with synthetic records representing a small event marketplace. The sample events represent different event categories and venues in the Los Angeles area: Indie Sounds Live at The Ford; Laugh Lines at The Ford; The Winter Play at Canyon Theater; and Summer Day Festival at Valley Park.

Table	Initial record count
Users	3
Venues	3
Events	4
TicketTypes	5
Orders	4
OrderItems	5
Payments	4

Table 4. Initial Seed-Data Counts

The order data intentionally contains multiple statuses, including confirmed, pending, and payment_failed. This allows the SQL queries to distinguish completed sales from orders that should not be included in confirmed ticket and revenue calculations. The passwords in the seed data are synthetic placeholders only. They are not intended to represent secure production credential storage.

4.6 Required SQL Queries

The application includes eight required SQL queries. Several queries join three or more tables, satisfying the requirement for multi-table relational analysis.

4.6.1 Query 1: Events at a Particular Venue

This query identifies events hosted at a specified venue.

```
SELECT e.*
FROM events AS e
JOIN venues AS v
   ON v.id = e.venue_id
WHERE v.name = 'The Ford'
ORDER BY e.date;
```

Expected result using the initial seed data

Event	Date	Venue
Indie Sounds Live	2026-11-14	The Ford
Laugh Lines	2026-12-05	The Ford

Table 5. Query 1 Expected Result — Events at The Ford

This query demonstrates a relational join between Events and Venues. It avoids storing repeated venue names inside every event record.

4.6.2 Query 2: Tickets Purchased by a Particular User

This query retrieves the ticket types and quantities purchased by a specific user.

```
SELECT
   u.username,
   e.name AS event_name,
   tt.name AS ticket_type,
   oi.quantity,
   oi.price AS price_per_ticket,
   oi.quantity * oi.price AS line_total
FROM users AS u
JOIN orders AS o ON o.user_id = u.id
JOIN order_items AS oi ON oi.order_id = o.id
JOIN ticket_types AS tt ON tt.id = oi.ticket_type_id
JOIN events AS e ON e.id = o.event_id
WHERE u.id = '00000000-0000-4000-8000-000000000001'
  AND o.status = 'confirmed'
ORDER BY e.date, tt.name;
```

Expected result using the initial seed data

Username	Event	Ticket type	Quantity	Price	Line total
alex.rivera	Indie Sounds Live	General Admission	2	$45.00	$90.00
alex.rivera	Indie Sounds Live	VIP Admission	1	$95.00	$95.00

Table 6. Query 2 Expected Result — Tickets Purchased by a User

This query joins five tables and demonstrates how normalized relational records can reconstruct a customer's purchase history.

4.6.3 Query 3: Total Tickets Sold for Each Event

This query counts only tickets associated with confirmed orders.

```
SELECT e.id AS event_id, e.name AS event_name,
    COALESCE(SUM(oi.quantity), 0) AS total_tickets_sold
FROM events AS e
LEFT JOIN orders AS o ON o.event_id = e.id AND o.status = 'confirmed'
LEFT JOIN order_items AS oi ON oi.order_id = o.id
GROUP BY e.id, e.name
ORDER BY total_tickets_sold DESC, e.name;
```

Expected result using the initial seed data

Event	Confirmed tickets sold
Indie Sounds Live	3
Laugh Lines	3
Summer Day Festival	0
The Winter Play	0

Table 7. Query 3 Expected Result — Confirmed Tickets Sold

The use of LEFT JOIN ensures that events without confirmed sales remain visible with a value of zero.

4.6.4 Query 4: Remaining Ticket Inventory

This query calculates remaining inventory using venue capacity minus confirmed tickets sold.

```
SELECT e.id AS event_id, e.name AS event_name, v.capacity,
    v.capacity - COALESCE(SUM(oi.quantity), 0) AS remaining_inventory
FROM events AS e
JOIN venues AS v ON v.id = e.venue_id
LEFT JOIN orders AS o ON o.event_id = e.id AND o.status = 'confirmed'
LEFT JOIN order_items AS oi ON oi.order_id = o.id
WHERE e.id = :event_id
GROUP BY e.id, e.name, v.capacity;
```

For the initial Indie Sounds Live event:

Event	Venue capacity	Confirmed tickets sold	Remaining inventory
Indie Sounds Live	1,200	3	1,197

Table 8. Query 4 Expected Result — Remaining Ticket Inventory

The current SQL statement uses :event_id as a parameter placeholder. When demonstrating the query through the generic SQL endpoint, the placeholder should be replaced by the concrete event UUID or handled through an application-level parameter. The current implementation treats venue capacity as the available event inventory. A more advanced design could introduce a dedicated event-inventory or ticket-type-inventory structure.

4.6.5 Query 5: Total Revenue for Each Event

This query calculates revenue using completed payments only.

```
SELECT e.id AS event_id, e.name AS event_name,
    COALESCE(SUM(p.amount), 0) AS total_revenue
FROM events AS e
LEFT JOIN orders AS o ON o.event_id = e.id
LEFT JOIN payments AS p ON p.order_id = o.id AND p.status = 'completed'
GROUP BY e.id, e.name
ORDER BY total_revenue DESC, e.name;
```

Expected result using the initial seed data

Event	Completed revenue
Indie Sounds Live	$185.00
Laugh Lines	$105.00
Summer Day Festival	$0.00
The Winter Play	$0.00

Table 9. Query 5 Expected Result — Completed Revenue

The payment-status filter prevents pending and failed payments from being counted as revenue.

4.6.6 Query 6: Customers Who Purchased the Most Tickets

This query ranks customers by the number of tickets associated with confirmed orders.

```
SELECT u.id AS user_id, u.username, u.firstname, u.lastname,
    SUM(oi.quantity) AS total_tickets_purchased
FROM users AS u
JOIN orders AS o ON o.user_id = u.id
JOIN order_items AS oi ON oi.order_id = o.id
WHERE o.status = 'confirmed'
GROUP BY u.id, u.username, u.firstname, u.lastname
ORDER BY total_tickets_purchased DESC, u.username;
```

Expected result using the initial seed data

Username	Total confirmed tickets
alex.rivera	3
jordan.lee	3

Table 10. Query 6 Expected Result — Customers with the Most Tickets

Sam Patel is excluded because the current order associated with that user is pending rather than confirmed.

4.6.7 Query 7: Events Above a Sales Threshold

This query identifies events whose confirmed ticket sales exceed two tickets.

```
SELECT e.id AS event_id, e.name AS event_name,
    SUM(oi.quantity) AS tickets_sold
FROM events AS e
JOIN orders AS o ON o.event_id = e.id AND o.status = 'confirmed'
JOIN order_items AS oi ON oi.order_id = o.id
GROUP BY e.id, e.name
HAVING SUM(oi.quantity) > 2
ORDER BY tickets_sold DESC, e.name;
```

Expected result using the initial seed data

Event	Tickets sold
Indie Sounds Live	3
Laugh Lines	3

Table 11. Query 7 Expected Result — Events Above the Sales Threshold

This query demonstrates the use of GROUP BY and HAVING for aggregate filtering.

4.6.8 Query 8: Monthly Ticket Revenue

This query groups completed revenue by the month associated with the event date.

```
SELECT DATE_TRUNC('month', e.date)::DATE AS revenue_month,
    SUM(p.amount) AS total_revenue
FROM payments AS p
JOIN orders AS o ON o.id = p.order_id
JOIN events AS e ON e.id = o.event_id
WHERE p.status = 'completed'
GROUP BY DATE_TRUNC('month', e.date)
ORDER BY revenue_month;
```

Expected result using the initial seed data

Revenue month	Total revenue
2026-11-01	$185.00
2026-12-01	$105.00

Table 12. Query 8 Expected Result — Monthly Ticket Revenue

Because the current Payments table does not include a payment timestamp, this query represents revenue by event month rather than the actual calendar month in which payment was processed.

4.7 Transaction Processing

The POST /orders endpoint demonstrates transaction support using a SQLAlchemy session. The implementation is located in api/services/orders_router.py. The transaction follows this sequence:

1. Create a database session.
2. Generate an order identifier if one is not supplied.
3. Add the order to the session.
4. Assign order identifiers to each order item.
5. Generate order-item identifiers when needed.
6. Add the order items to the same session.
7. Commit the session.
8. Roll back the session if an exception occurs.
9. Close the session.

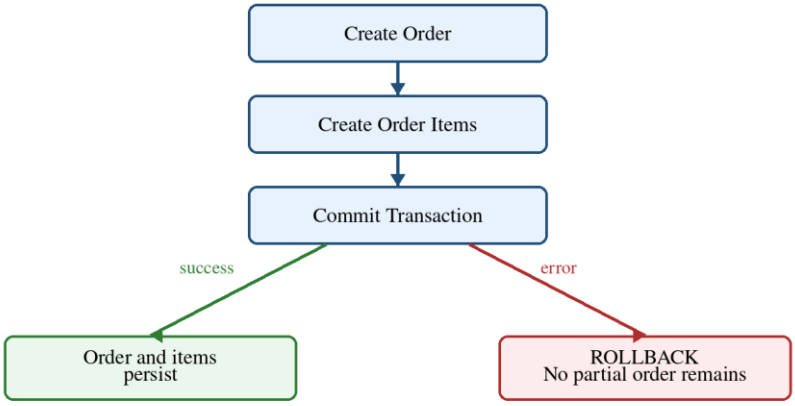


Figure 2. PostgreSQL order transaction and rollback flow.

Transaction support is important because an order without its associated order items would be incomplete. Likewise, order items without a corresponding order would create orphaned purchase records. A single transaction ensures that the order and its items are treated as one atomic unit. The current implementation protects order and order-item creation. It does not yet include payment creation or an explicit inventory update within the same transaction. Therefore, the report should describe this as a meaningful order-composition transaction rather than as a complete production ticket-purchase transaction.

4.8 Transaction Demonstration

Successful transaction

A successful demonstration should create a new pending order containing at least one order item. Using a pending status prevents the demonstration order from changing confirmed ticket-sales and revenue results. After the API returns successfully, the database should be queried to verify:

- The order exists.
- The order item exists.
- The order item references the created order.
- The quantity and price were stored correctly.

Rollback demonstration

A rollback demonstration should deliberately create a database failure after the order and at least one order item have been staged. One practical approach is to submit two order items with the same nonzero primary-key identifier. The duplicate identifier should cause the transaction to fail when the session is committed. The verification query should confirm that:

- The API returned an error.
- The order was not persisted.
- The order items were not persisted.
- No partial records remain.

4.9 Monetary Data and Relational Limitations

The current implementation uses DOUBLE PRECISION for prices and monetary amounts. This is acceptable for demonstrating the database architecture, but production financial systems generally use fixed-precision numeric types such as NUMERIC(10,2) to avoid floating-point representation issues. The current schema also lacks explicit foreign-key constraints for several application-level relationships. These limitations should be discussed in Section 11 and, if the schema is updated, reflected in the final ER diagram and schema evidence.

4.10 PostgreSQL Schema Evidence

The final report should include one PostgreSQL schema screenshot after this section. The screenshot should show the seven required tables and make the primary keys, NOT NULL attributes, unique constraints, indexes, and currently enforced foreign keys readable. This evidence should be taken from the PostgreSQL implementation defined in postgres/init.sql. No additional SQL-query screenshots are required for the final report; the query definitions and expected results are documented above.

5. MongoDB Design

5.1 Role of MongoDB

MongoDB stores flexible and semi-structured event information, while PostgreSQL remains authoritative for users, venues, events, ticket types, orders, order items, and payments. The MongoDB collection is event_content in the group4db database. Its documents enrich relational events with descriptions, speakers, schedules, reviews, and tags without adding a separate relational table for every variation.

Database	Primary responsibility	Examples
PostgreSQL	Structured transactional data	Events, orders, payments, inventory calculations
MongoDB	Flexible event enrichment	Descriptions, speakers, schedules, reviews, tags
Redis	Temporary and derived high-speed data	Event cache and trending scores

Table 13. Data ownership across EventSphere databases

5.2 MongoDB Document Model

Each event-content document combines common identifiers with nested objects and arrays. The eventId value is a BSON UUID that corresponds to events.id in PostgreSQL. The capitalized field names reflect the current implementation.

```
{
  eventId: UUID("20000000-0000-4000-8000-000000000001"),
  Genre: "Indie Rock",
  Description: { summary: "...", ageRestriction: "All ages", accessibility: [...] },
  Speakers: [{ name: "Maya Chen", role: "Singer-songwriter", topic: "..." }],
  Schedule: [{ startTime: "18:00", endTime: "19:00", title: "Doors open" }],
  Reviews: [{ reviewer: "Alex Rivera", rating: 5, comment: "..." }],
  Tags: ["indie", "live-music", "concert"]
}
```

The structure is consistent enough for common retrieval but flexible enough for event-specific attributes. Music events can include accessibility information, comedy events can include language or age restrictions, theater events can include content advisories, and festivals can include amenities.

5.3 Collection Initialization and Shared Identifiers

The collection and seed records are defined in mongo/mongo-init.js. The initialization creates the event_content collection, adds a unique ascending index on eventId, inserts three documents with insertMany(), and inserts a fourth document with insertOne(). The collection therefore contains one content document for each of the four PostgreSQL events.

Event	Shared identifier	Flexible content
Indie Sounds Live	...0001	Indie Rock, speakers, schedule, reviews, tags
Laugh Lines	...0002	Stand-up content and age restriction
The Winter Play	...0003	Drama content and advisories
Summer Day Festival	...0004	Multi-genre content and amenities

Table 14. Shared PostgreSQL and MongoDB event identifiers

The application converts UUIDs using Binary.from_uuid(id, UuidRepresentation.STANDARD) in api/services/events_router.py. MongoDB does not enforce a foreign key to PostgreSQL; the shared UUID convention and the unique eventId index are maintained by the application and initialization process.

5.4 Required MongoDB Operations

Operation or query	Implemented form	Purpose or expected result
Insert	insertMany() and insertOne()	Create four event-content documents
Projection	{ _id: 0 }	Return application fields without MongoDB _id
Tag search	{ Tags: "concert" }	Returns Indie Sounds Live and Summer Day Festival
Speaker search	{ "Speakers.name": "Maya Chen" }	Returns Indie Sounds Live
Review comparison	Reviews.$elemMatch.rating $gte 5	Returns events with a five-star review

Operation or query	Implemented form	Purpose or expected result
Nested attributes	Description.ageRestriction and Speakers.$elemMatch	Search nested objects and the same array element
Boolean search	$and and $or	Combine tag, genre, and nested conditions
Mutation	$push and $pull on Reviews	Add and remove review subdocuments

Table 15. MongoDB operation and query coverage

The eight query forms are exposed through the tag, speaker, review, attributes, and concert-genre search routes. The parameterized attributes route produces distinct searches for minimum review rating, speaker role and topic, age restriction, and combined tag and age restriction. Together, these operations demonstrate array membership, dot notation, comparison operators, Boolean operators, $elemMatch, projection, updates, deletes, and indexing.

5.5 Cross-Database Retrieval and Mutations

GET /events/{id}/content retrieves the MongoDB document using the shared UUID and returns HTTP 404 when no document is found. The general event-detail route first obtains the relational event from PostgreSQL, retrieves matching event content from MongoDB, combines the two records, and optionally stores the result in Redis. This is application-level composition rather than a direct database join.

POST /events/{id}/reviews -> update_one({ eventId: id }, { $push: { Reviews: review } })

DELETE /events/{id}/reviews/{reviewer} -> update_one({ eventId: id }, { $pull: { Reviews: { reviewer } } })

A temporary reviewer should be used during demonstrations so the seeded reviews remain available. The Angular API Playground exposes event-content retrieval, review insertion and deletion, and the search routes through the /api proxy. It displays the response status, JSON payload, and elapsed request time.

5.6 Design Justification and Limitations

MongoDB is appropriate because event metadata varies by event type and nested documents represent speakers, schedule entries, and reviews naturally. Arrays preserve related content in one document, while expressive searches operate over nested fields. MongoDB is not the authority for inventory, payment status, order completion, or confirmed sales; those remain in PostgreSQL.

The main limitations are application-managed cross-database identity, inconsistent capitalization in field names, limited indexing beyond eventId, and possible initialization-script nonexecution when an existing Docker volume is reused. The final evidence should verify the collection count, unique eventId index, representative document, and concrete executions of the required query forms.

6. Redis Design

6.1 Role and Configuration

Redis is a supporting data store for low-latency derived information. PostgreSQL remains authoritative for transactions, payments, and inventory; MongoDB remains authoritative for flexible event content. The Redis service is defined in docker-compose.yml and uses redis:7.4-alpine, port 6379, the redis_data volume, and append-only persistence. The API reads REDIS_HOST, uses database 0, and applies one-second connection and socket timeouts so Redis failure does not indefinitely block event retrieval.

Use case	Redis structure and key	Command behavior
Event-response cache	String; event:{event_id}	SET serialized JSON with ex=60; GET on retrieval
Trending events	Sorted set; trending:events	ZINCRBY on valid views; ZREVRANGE for top ten

Table 16. Redis structures and command responsibilities

6.2 Event-Response Cache

The event endpoint uses a cache-aside pattern. A request for GET /events/{id} checks Redis first. On a valid hit, the serialized response is parsed and returned. On a miss, or when bypass_cache=true is supplied, FastAPI retrieves the structured event from PostgreSQL and flexible content from MongoDB, combines the records, and stores the JSON response for 60 seconds unless caching is bypassed.

```
cache_key = f"event:{event_id}"
redis_client.set(cache_key, json.dumps(jsonable_encoder(response)), ex=60)
cached_event = redis_client.get(cache_key)
GET /events/{id}?bypass_cache=true  # database-backed comparison path
```

A cache hit and a database-backed request both increment the event's trending score when the event exists. Invalid cached JSON is deleted and rebuilt. Redis read or write failures are treated as degraded-supporting-service conditions, allowing the database-backed response to continue when possible.

6.3 Trending Sorted Set

The sorted set trending:events stores event UUIDs as members and successful event-detail view counts as scores. ZINCRBY updates a score atomically, while ZREVRANGE 0 9 WITHSCORES returns the ten highest-ranked members. GET /trending returns identifiers and scores rather than full event documents; clients can request details separately through GET /events/{id}.

```
redis_client.zincrby("trending:events", 1, str(event_id))
redis_client.zrevrange("trending:events", 0, 9, withscores=True)
```

6.4 Demonstration and Performance

1. Request the known event with bypass_cache=true to show the database-backed path.
2. Request the same event without the bypass parameter to populate event:{id}.
3. Repeat the request to demonstrate a cache hit and compare elapsed time.
4. Verify GET event:{id}, TTL event:{id}, ZREVRANGE trending:events 0 9 WITHSCORES, and GET /trending.

Metric	Database-backed request	Redis-cached request
Minimum	11.269 ms	2.802 ms
Maximum	16.087 ms	4.339 ms
Average	13.273 ms	3.308 ms
Median	12.590 ms	3.248 ms

Table 17. Redis performance summary from the ten-request comparison

The recorded average indicates that the cached path was approximately 4.01 times faster and 75.1 percent lower in response time. The difference is consistent with one Redis lookup replacing sequential PostgreSQL and MongoDB reads plus response composition. The raw measurements and method should be presented again in Section 9 with the benchmark command or terminal evidence.

6.5 Consistency, Reliability, and Limitations

The 60-second TTL bounds normal cache staleness, but review updates and deletes do not explicitly invalidate event:{id}; a changed review can therefore remain stale until expiration. Other limitations are an indefinitely growing trending sorted set, no demonstrated replication, failover, or authentication, and no dedicated benchmark

script. Event retrieval degrades to PostgreSQL and MongoDB when Redis fails, while /trending returns an error when its ranking cannot be retrieved.

7. Application Integration

EventSphere integrates an Angular frontend, an Nginx reverse proxy, a FastAPI application layer, PostgreSQL, MongoDB, Redis, and Docker Compose into one local environment. The application layer serves as the integration boundary. The browser communicates with FastAPI through HTTP endpoints, while FastAPI selects the appropriate database for each operation. The complete containerized environment is defined in docker-compose.yml, which separates the frontend, backend, and database services while allowing them to communicate through Docker service names.

7.1 FastAPI Application Boundary

The main FastAPI application is defined in api/main.py. It registers four router groups: Events, Users, Orders, and SQL execution. This structure provides a single HTTP boundary for the frontend while keeping each functional area in a separate service module. The events router coordinates PostgreSQL event records, MongoDB event-content documents, and Redis caching and trending operations. These cross-database responsibilities are implemented in api/services/events_router.py.

The order router manages order creation and retrieval through PostgreSQL. The POST /orders operation creates an order and its associated order items within one SQLAlchemy session. This behavior is implemented in api/services/orders_router.py. FastAPI also provides request validation, error handling, response serialization, and status-code management, so the frontend does not need to understand database-specific connection details or query syntax.

7.2 Angular API Playground

The Angular frontend is implemented in web/src/app/app.ts. It provides an API Playground organized into Events, Users, Orders, and Database groups. The current interface exposes 17 endpoint cards for event retrieval, MongoDB content, reviews, searches, trending events, users, orders, and SQL execution.

The frontend dynamically builds requests from editable path parameters, query parameters, and JSON request bodies. It prefixes requests with /api, encodes path values, and serializes JSON bodies before sending them to the backend. For every request, the interface records the HTTP status, success state, response payload, and elapsed time. This makes the database operations observable through the running application rather than requiring source-code or command-line interaction alone.

7.3 Reverse-Proxy and Container Integration

The browser communicates with the Angular application through the web container on port 8096. Nginx serves the compiled frontend files and forwards requests beginning with /api/ to the FastAPI container on port 8890. The service definitions, ports, environment variables, and internal hostnames are specified in docker-compose.yml.

Service	Main responsibility	Interface
comp642-web	Serves the Angular API Playground through Nginx	Port 8096
comp642-api	Hosts the FastAPI application and routers	Port 8890
local-postgres	Stores structured transactional data	Port 5432
local-mongodb	Stores flexible event-content documents	Port 27017
local-redis	Stores event caches and trending scores	Port 6379

Table 18. EventSphere application and service integration

7.4 End-to-End Request Flow

A typical event-detail request follows this sequence: the user selects an endpoint in the Angular API Playground; Angular sends a request beginning with /api; Nginx forwards it to FastAPI; FastAPI validates the request and invokes the appropriate router; the router retrieves data from PostgreSQL, MongoDB, or Redis; FastAPI formats the result as JSON; and Angular displays the status, response payload, and elapsed time.

For GET /events/{id}, the events router can check Redis first. If the cached response is unavailable, it retrieves the structured event from PostgreSQL, retrieves matching content from MongoDB, combines the records, and optionally stores the result in Redis. The frontend receives one integrated response even though multiple databases may have been accessed. For POST /orders, the request body is validated by FastAPI and the orders router creates the order and order items in PostgreSQL as one transaction.

7.5 Integration Assessment

The integration design separates responsibilities clearly. Angular is responsible for interaction and demonstration, Nginx serves the frontend and forwards API traffic, FastAPI coordinates requests, and each database manages the type of data for which it is most appropriate. Database connection details remain in the backend, while the frontend communicates through stable HTTP endpoints. The same API contract can therefore support the Angular demonstration interface and future clients.

The design remains primarily a local demonstration architecture. Docker Compose defines service dependencies, but it does not include database health checks, production authentication, or high-availability mechanisms. These limitations do not prevent the project from demonstrating application integration, but they distinguish the prototype from a production deployment.

8. Cross-Database Operation

EventSphere coordinates PostgreSQL, MongoDB, and Redis through application-level workflows rather than treating them as one distributed database. Each store keeps a clear responsibility: PostgreSQL owns structured and transactional records, MongoDB owns flexible event content, and Redis stores derived or temporary values. This approach keeps the design understandable while making the cross-database behavior visible through the API.

8.1 Composite Event Retrieval

The main cross-database read is GET /events/{id}, implemented in api/services/events_router.py. The route first attempts to read the combined response from Redis. On a cache miss, or when bypass_cache=true is used, it retrieves the structured event from PostgreSQL, retrieves the matching event_content document from MongoDB using the shared UUID, combines the records, and optionally stores the resulting JSON response in Redis for 60 seconds. The client therefore receives one event response even though the data originates from multiple databases.

8.2 Cross-Database Ownership and Mutations

The system deliberately limits each mutation to the database that owns the affected data. Review insertion and deletion update MongoDB arrays with $push and $pull. Order creation updates PostgreSQL orders and order_items inside one SQLAlchemy transaction, as implemented in api/services/orders_router.py. Redis is not used as a source of truth for either operation; it provides the event cache and the derived trending score.

Operation	PostgreSQL	MongoDB	Redis
Event detail	Authoritative event row	Matching event_content document	Optional response cache and view score
Review update	No write	$push or $pull in Reviews	Existing cache may remain stale until TTL
Order creation	Order and items commit or roll back	No write	No write
Trending request	No write	No write	ZINCRBY and ZREVRANGE

Table 19. Cross-database operation responsibilities

8.3 Consistency Boundary

The shared event UUID is the correlation key across the three stores, but the application—not PostgreSQL or MongoDB—maintains that relationship. The architecture therefore provides application-level composition rather than a distributed transaction. Redis failures are treated as degraded-supporting-service conditions for event retrieval, allowing the API to fall back to PostgreSQL and MongoDB. The principal limitation is that a multi-store workflow can still experience partial completion unless it is protected by explicit invalidation, retry, idempotency, or an event-driven coordination mechanism.

9. Performance Experiment

9.1 Experiment Design

The experiment compares two paths for retrieving the same known event: a database-backed request using bypass_cache=true and a Redis-cached request using the normal event endpoint. The comparison documented in the project README.md uses ten requests for each path in the local Docker environment. The event identifier, response

type, and application environment remain constant; the independent variable is whether Redis serves the combined response. The database-backed path performs sequential PostgreSQL and MongoDB reads, while the cached path performs a Redis lookup and JSON deserialization.

9.2 Results and Interpretation

The measurements in Table 17 show a minimum of 11.269 ms, a maximum of 16.087 ms, an average of 13.273 ms, and a median of 12.590 ms for the database-backed path. The Redis-cached path records a minimum of 2.802 ms, a maximum of 4.339 ms, an average of 3.308 ms, and a median of 3.248 ms. The average speed-up is approximately 13.273 / 3.308 = 4.01 times, and the average response time is approximately 75.1 percent lower with Redis.

The result is consistent with the architecture: a cache hit avoids the sequential database reads and response-composition work required on a miss. It demonstrates a meaningful benefit for repeated event-detail requests, but it is not a production capacity claim. The sample is small, local, and likely reflects a warm application and warm databases.

9.3 Validity and Reproducibility

A stronger experiment would repeat the comparison with cold and warm caches, larger datasets, concurrent clients, repeated trials, and percentile metrics such as p95 and p99. It should also record CPU, memory, database connection-pool behavior, and cache-eviction events. The current measurements are appropriate for demonstrating the performance implication of the implemented Redis path, while the limitations should be stated when interpreting the result.

10. Architecture Analysis

10.1 Architectural Strengths

- Polyglot persistence is used for a reason: PostgreSQL supports relationships and transactions, MongoDB supports variable nested content, and Redis supports low-latency derived access.
- FastAPI provides one application boundary, so the Angular client does not need to manage database drivers, UUID conversions, transaction rules, or cache commands.
- The API Playground makes the behavior observable by showing request status, response JSON, and elapsed time for the database operations.
- Redis is treated as an optimization and a derived ranking store, which prevents cache loss from becoming loss of authoritative orders, payments, or event records.

10.2 Trade-Offs and Scalability

The principal trade-off is operational complexity. Three data models require separate initialization, monitoring, backups, failure handling, and consistency practices. The shared UUID convention simplifies correlation but does not replace foreign keys or distributed transactions. The stateless API boundary can be scaled behind the reverse proxy, while PostgreSQL indexes, the MongoDB eventId index, and Redis key-based access provide efficient paths for the current workload. At larger scale, the design would need connection-pool tuning, health checks, asynchronous work for non-critical derived updates, and a clearer strategy for cache invalidation.

10.3 Security and Deployment Readiness

The local deployment described in docker-compose.yml is suitable for demonstration and evaluation, not for production exposure. The next maturity steps are secret management, authentication and authorization, TLS, restricted database networking, database health checks, structured logging, backups, replication, and failure recovery. These changes would strengthen the operational architecture without changing the basic data-ownership decisions demonstrated by EventSphere.

11. Challenges and Limitations

The project demonstrates the required behavior, but the implementation also exposes limitations that are important for an honest architectural assessment.

- Several relationships are represented by the application and logical model but are not yet enforced with PostgreSQL foreign-key constraints. Adding those constraints would improve referential integrity and make the physical schema match the logical ER model more completely.

• Prices and monetary amounts use DOUBLE PRECISION. A production implementation should use NUMERIC with an explicit scale, such as NUMERIC(10,2), to avoid floating-point representation issues.
• MongoDB field names use capitalization such as Genre, Description, Speakers, Reviews, and Tags. A consistent naming convention and additional indexes would improve maintainability as the collection grows.
• Review updates and deletes do not explicitly invalidate the event cache, so a response can remain stale until the 60-second TTL expires. The trending sorted set also has no demonstrated expiration or rebuild process.
• The system does not implement a distributed transaction across all three databases. The current design intentionally protects the PostgreSQL order composition transaction and treats Redis as recoverable derived state.
• The initialization scripts depend on database-volume state. When an existing Docker volume is reused, the team must verify the actual collection, indexes, tables, and seed counts instead of assuming initialization ran again.
• The performance comparison uses a small synthetic dataset and ten-request measurements. It does not establish production throughput, concurrency limits, or failure behavior under load.
• The local Docker Compose environment does not demonstrate production authentication, high availability, database replication, or automated health checks. These omissions define the boundary between the course prototype and a deployable service.

12. Conclusions

EventSphere demonstrates a coherent multi-database architecture for an event marketplace. PostgreSQL provides the structured system of record and transaction boundary; MongoDB provides flexible event enrichment with nested documents and arrays; and Redis provides a cache-aside path and a sorted-set ranking for trending events. The shared UUID convention and FastAPI service layer connect these technologies without obscuring their different responsibilities.

The application integration is visible through the Angular API Playground, Nginx reverse proxy, FastAPI routers, and Docker Compose environment. The cross-database event-detail operation shows how data can be composed at the application boundary, while the order endpoint shows why transactional data should remain in PostgreSQL. The performance experiment further demonstrates that Redis can substantially reduce latency for repeated reads, while the analysis identifies the consistency and operational costs of that optimization.

Overall, the project meets the architectural objective by showing not only how to issue database commands, but also how to assign data to appropriate storage models, coordinate those models through an API, evaluate the resulting trade-offs, and identify the changes required for production readiness. The remaining improvements are clear: strengthen relational constraints and monetary types, formalize cache invalidation, improve initialization and observability, and repeat performance testing under realistic workload conditions.